

4.6.4

Taraka Abhinav - EE25BTECH11016

October 3, 2025

Question

Find the equation of a line passing through the point $(2, 3, 2)$ and parallel to the line:

$$\mathbf{r} = \begin{pmatrix} -2 \\ 3 \\ 0 \end{pmatrix} + \lambda \begin{pmatrix} 2 \\ -3 \\ 6 \end{pmatrix}$$

Also, find the distance between these two lines.

Step 1: Equation of the Required Line

The vector equation of a line is $\mathbf{r} = \mathbf{a}_2 + \mu\mathbf{b}$.

The line passes through the point $(2, 3, 2)$, so the position vector is:

$$\mathbf{a}_2 = \begin{pmatrix} 2 \\ 3 \\ 2 \end{pmatrix} \quad (1)$$

The line is parallel to the given line, so they share the same direction vector:

$$\mathbf{b} = \begin{pmatrix} 2 \\ -3 \\ 6 \end{pmatrix} \quad (2)$$

The equation of the required line is:

$$\mathbf{r} = \begin{pmatrix} 2 \\ 3 \\ 2 \end{pmatrix} + \mu \begin{pmatrix} 2 \\ -3 \\ 6 \end{pmatrix} \quad (3)$$

Step 2: Distance via Least Squares

Let $\Delta \mathbf{a} = \mathbf{a}_2 - \mathbf{a}_1$. We want to find the minimum of the squared distance function $f(\nu)$:

$$\begin{aligned} f(\nu) &= \|\Delta \mathbf{a} - \nu \mathbf{b}\|^2 \\ &= (\Delta \mathbf{a} - \nu \mathbf{b})^T (\Delta \mathbf{a} - \nu \mathbf{b}) \\ &= \Delta \mathbf{a}^T \Delta \mathbf{a} - 2\nu (\Delta \mathbf{a}^T \mathbf{b}) + \nu^2 (\mathbf{b}^T \mathbf{b}) \end{aligned}$$

To find the minimum, we set the derivative with respect to ν to zero.

Step 3: Minimizing the Distance

Taking the derivative of $f(\nu)$ and setting it to zero:

$$\frac{df}{d\nu} = -2(\Delta \mathbf{a}^T \mathbf{b}) + 2\nu(\mathbf{b}^T \mathbf{b}) = 0 \quad (4)$$

Solving for the optimal value of ν :

$$\nu_{opt} = \frac{\Delta \mathbf{a}^T \mathbf{b}}{\mathbf{b}^T \mathbf{b}} \quad (5)$$

Substituting this back into the distance function gives the minimum distance squared.

Step 4: Final Distance Formula

The minimum squared distance is:

$$d^2 = \frac{(\Delta \mathbf{a}^T \Delta \mathbf{a})(\mathbf{b}^T \mathbf{b}) - (\Delta \mathbf{a}^T \mathbf{b})^2}{\mathbf{b}^T \mathbf{b}} \quad (6)$$

Using Lagrange's identity, the numerator simplifies to $\|\Delta \mathbf{a} \times \mathbf{b}\|^2$.

$$d^2 = \frac{\|\Delta \mathbf{a} \times \mathbf{b}\|^2}{\|\mathbf{b}\|^2} \implies d = \frac{\|\Delta \mathbf{a} \times \mathbf{b}\|}{\|\mathbf{b}\|} \quad (7)$$

Step 5: Numerical Calculation

We substitute the numerical values:

$$\Delta \mathbf{a} = \begin{pmatrix} 4 \\ 0 \\ 2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 2 \\ -3 \\ 6 \end{pmatrix} \quad (8)$$

The cross product is:

$$\Delta \mathbf{a} \times \mathbf{b} = \begin{pmatrix} 6 \\ -20 \\ -12 \end{pmatrix} \quad (9)$$

The norms are:

$$\|\Delta \mathbf{a} \times \mathbf{b}\| = \sqrt{6^2 + (-20)^2 + (-12)^2} = \sqrt{580} \quad (10)$$

$$\|\mathbf{b}\| = \sqrt{2^2 + (-3)^2 + 6^2} = 7 \quad (11)$$

The distance is:

$$d = \frac{\sqrt{580}}{7} = \frac{2\sqrt{145}}{7} \quad (12)$$

```
#include <stdio.h>
#include <math.h>

int main() {
    double a1[3] = {-2, 3, 0};
    double a2[3] = {2, 3, 2};
    double b[3] = {2, -3, 6};
    double da[3], v[3];

    for (int i = 0; i < 3; i++) da[i] = a2[i] - a1[i];
```

```
v[0] = b[1]*da[2] - b[2]*da[1];  
v[1] = b[2]*da[0] - b[0]*da[2];  
v[2] = b[0]*da[1] - b[1]*da[0];  
  
double vnorm = sqrt(v[0]*v[0] + v[1]*v[1] + v[2]*v[2]);  
double bnorm = sqrt(b[0]*b[0] + b[1]*b[1] + b[2]*b[2]);  
  
double d = vnorm / bnorm;  
printf("Distance = %lf\n", d);  
  
return 0;  
}
```

Python + C Code

```
import ctypes
import numpy as np
import matplotlib.pyplot as plt

# Load compiled shared library
lib = ctypes.CDLL('./line_distance.so')
lib.main()

# Given data
a1 = np.array([-2, 3, 0])
a2 = np.array([2, 3, 2])
b = np.array([2, -3, 6])

# Compute distance
da = a2 - a1
v = np.cross(b, da)
d = np.linalg.norm(v) / np.linalg.norm(b)
print("Distance =", d)
```

```
# Projection of da onto b to find perpendicular foot
```

```
proj = (np.dot(da, b) / np.dot(b, b)) * b
```

```
perp_vec = da - proj
```

```
p1 = a1 + proj
```

```
p2 = a2
```

```
# Plot
```

```
fig = plt.figure()
```

```
ax = fig.add_subplot(111, projection='3d')
```

```
# Line 1 and Line 2
```

```
t = np.linspace(-2, 2, 10)
```

```
line1 = a1 + np.outer(t, b)
```

```
line2 = a2 + np.outer(t, b)
```

```
ax.plot(line1[:,0], line1[:,1], line1[:,2], 'r', label='Line 1')
```

```
ax.plot(line2[:,0], line2[:,1], line2[:,2], 'b', label='Line 2')
```

```
# Points with vector labels
ax.scatter(*a1, color='red')
ax.text(a1[0], a1[1], a1[2], f"A1 {a1}", color='red', fontsize
        =10)
ax.scatter(*a2, color='blue')
ax.text(a2[0], a2[1], a2[2], f"A2 {a2}", color='blue', fontsize
        =10)

# Perpendicular distance line
ax.plot([p1[0], p2[0]], [p1[1], p2[1]], [p1[2], p2[2]], 'g--',
        label=f'Distance = {round(d,3)}')

ax.legend()
plt.show()
```

Python Plotting Code

```
import numpy as np
import matplotlib.pyplot as plt

# Data
a1 = np.array([-2, 3, 0])
a2 = np.array([2, 3, 2])
b = np.array([2, -3, 6])

# Distance calculation
da = a2 - a1
v = np.cross(b, da)
d = np.linalg.norm(v) / np.linalg.norm(b)
print("Distance =", d)

# Projection for perpendicular line
proj = (np.dot(da, b) / np.dot(b, b)) * b
p1 = a1 + proj
p2 = a2
```

Python Plotting Code

```
# Plot
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

# Lines
t = np.linspace(-2, 2, 10)
line1 = a1 + np.outer(t, b)
line2 = a2 + np.outer(t, b)

ax.plot(line1[:,0], line1[:,1], line1[:,2], 'r', label='Line 1')
ax.plot(line2[:,0], line2[:,1], line2[:,2], 'b', label='Line 2')

# Points with labels
ax.scatter(*a1, color='red')
ax.text(a1[0], a1[1], a1[2], f"A1 {a1}", color='red', fontsize
        =10)
ax.scatter(*a2, color='blue')
ax.text(a2[0], a2[1], a2[2], f"A2 {a2}", color='blue', fontsize
        =10)
```

Python Plotting Code

```
# Perpendicular distance line
ax.plot([p1[0], p2[0]], [p1[1], p2[1]], [p1[2], p2[2]], 'g--',
        label=f'Distance = {round(d,3)}')

# Direction vectors (as arrows from A1 and A2)
ax.quiver(a1[0], a1[1], a1[2], b[0], b[1], b[2], color='red',
          arrow_length_ratio=0.1, linewidth=1.5)
ax.text(a1[0]+b[0]*0.5, a1[1]+b[1]*0.5, a1[2]+b[2]*0.5, "b",
        color='red')

ax.quiver(a2[0], a2[1], a2[2], b[0], b[1], b[2], color='blue',
          arrow_length_ratio=0.1, linewidth=1.5)
ax.text(a2[0]+b[0]*0.5, a2[1]+b[1]*0.5, a2[2]+b[2]*0.5, "b",
        color='blue')

# Make plot readable
ax.legend()
ax.set_xlabel("X")
ax.set_ylabel("Y")
```

Plot of the Parallel Lines

